

FitTrackr : plateforme web de suivi sportif et e-commerce

Université Paris Nanterre



Étudiants :



41019130
Salah-eddine Bekkari



35006249
Erick Jana Guana



43019983
Sarah Metref



43006475
Yacine

Dépôt Git du projet :

<https://github.com/salahbekkaripro/FitTrackr>

Instance déployée (Render) :

<https://fittrackr-fgq9.onrender.com>

Table des Matières

1. Introduction et Langages utilisés	3
2. Contexte et Objectifs du projet	4
3. Bibliothèques, Outils et Technologies	5
4. Travail réalisé	6
(a) (a) Organisation	7
5. Tests et non-régression	10
6. Difficultés rencontrées	11
(a) (a) Partie 1	11
(b) (b) Partie 2	12
7. Bilan	12
(a) (a) Conclusion	13
(b) (b) Perspectives	13
8. Exemple d'exécution	15
9. Manuel utilisateur	17
10. Webographie	14
11. Annexes	15
(a) (a) Cahier des charges	15

Bonne lecture et exploration

1 Introduction et Langages utilisés

Introduction

FitTrackr est une application web complète de suivi sportif. Elle combine une authentification personnalisée, un module d'onboarding pour collecter l'âge, le poids et la taille, un catalogue d'exercices, la planification de séances, le suivi des abonnements et un mini espace boutique. L'architecture repose sur Django 5 et une séparation par applications (`core`, `programs`, `shop`) afin d'isoler les responsabilités. La logique côté serveur (formulaires, validation, modèle de données) garantit la cohérence métier, tandis que l'interface HTML/CSS assure la lisibilité et la sécurité (aucun calcul métier n'est effectué côté client). Les vues ont été pensées pour rester simples et testables : pas d'API externe, mais des appels internes sécurisés par le framework.

Langages et outils utilisés

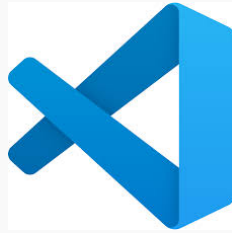
Langages et frameworks :

- **Python 3.12 / Django 5** : coeur backend, modèles relationnels, migrations, middlewares d'authentification et système de messages. Django a été choisi pour ses garde-fous (CSRF, XSS, gestion des sessions) et son modèle de données qui permet de changer de SGBD sans réécrire les requêtes. Les formulaires (`UserCreationForm`, `ModelForm`) apportent la validation serveur et la sérialisation des entrées.
- **HTML5 / CSS3 / JavaScript léger** : gabarits Django pour générer les vues, grille responsive pour les pages programmes et boutique, interactions minimales (sélection de quantités, filtres de catégories). Le choix d'un JS réduit limite la surface d'attaque et maintient la performance sur mobile.
- **SQL (SQLite en développement)** : stockage local pour itérer rapidement, avec des contraintes (clés étrangères, unicité) activées. L'usage du modèle relationnel facilite une bascule simple vers PostgreSQL en production.

1.1 Outils

Outils

Visual Studio Code (VS Code) : extensions Python et Django pour la compl  tion des mod  les, v  rification des imports et navigation rapide vers les templates. Le d  bogueur int  gr   a servi    inspecter les formulaires et la session utilisateur pendant l'onboarding.



Git : historique structur   par fonctionnalit  s (abonnements, programmes, boutique) avec relectures crois  es. Le versionnement des migrations a   t   essentiel pour figer le sch  ma au m  me rythme que les mod  les Python et   viter les r  gressions sur le mod  le utilisateur personnalis  .



2 Contexte et Objectifs du projet

Contexte du projet

FitTrackr cible un besoin r  current : centraliser le suivi sportif (objectifs, s  ances, poids) tout en proposant des parcours diff  renci  s (membres, coachs, administrateurs) et un canal de mon  tisation (abonnements et boutique). Le choix d'un framework monolithique mais modulaire (Django) permet de ma  triser la coh  rence des donn  es sans multiplier les services. Le p  rim  tre couvre la gestion du cycle de vie d'un utilisateur : inscription, collecte de donn  es de sant  , choix d'un abonnement avec engagements, planification de programmes, enregistrement des s  ances et achats d'accessoires.

Objectifs du projet

- **Définir un socle d'authentification robuste** : modèle utilisateur personnalisé avec rôles (member, coach, admin) et email unique. Justification : préparer des permissions fines (accès admin aux comptes, coachs aux programmes) sans dupliquer les tables.
- **Mettre en place un onboarding contextualisé** : collecte d'âge, poids, taille et objectif pondéral via OnboardingForm. Cela réduit l'attrition en guidant immédiatement l'utilisateur vers un programme adapté.
- **Structurer la donnée fitness** : tables Workout, WorkoutSet, Exercise, Program avec relations fortes et contraintes d'intégrité. Le modèle favorise les requêtes analytiques (temps passé, volume soulevé) et la recommandation future.
- **Gérer les engagements et upgrades** : logique métier dans subscriptions_view pour autoriser uniquement les upgrades pendant un engagement actif. Alternative envisagée : un service de facturation externe; rejeté pour limiter la complexité et rester hors-ligne.
- **Créer un tunnel d'achat simple** : panier, validation d'adresse, pseudo-paiement et historique. L'approche "server-side only" évite les appels API externes et limite les failles type injection JS.

3 Bibliothèques, Outils et Technologies

Bibliothèques utilisées

Chaque dépendance a été retenue pour un apport précis au produit et un coût de maintenance maîtrisé.

- **Django 5.2.9** : fournit le modèle de données, la couche de vues, le routage, la sécurité CSRF et les formulaires. Justification : cadre complet réduisant la quantité de code "maison" et offrant des migrations cohérentes. Limite : moins flexible qu'une stack microservices pour la scalabilité horizontale.
- **Pillow** : gestion des images produits (upload vers media/products) avec stockage local pour rester simple en développement.
- **sqlite3 (via le modèle Django)** : base légère pour itérations rapides. Limite : concurrence limitée; bascule prévue vers PostgreSQL pour la prod avec le même code de modèle.
- **typing_extensions / sqlparse** : support du typage et parsing SQL utilisés en interne par Django; ces dépendances renforcent la robustesse des migrations et des validations.

Remarque

Les bibliothèques ont été sélectionnées pour maximiser la sécurité par défaut (CSRF, validation des formulaires), la maintenabilité (migrations versionnées) et la portabilité du modèle de données. Un changement d'architecture reste possible : FastAPI + React offrirait plus d'interactivité mais exigerait la réécriture de la logique formulaire et des vues.

3.1 Technologies et Outils d'Assistance

Les outils IA au service du delivery

Pour accélérer l'itération, nous avons ponctuellement sollicité des LLMs (ChatGPT, Mistral) pour :

- **Prototyper des requêtes** : vérifier la bonne utilisation de `select_related/prefetch_related` sur les historiques de commandes et de séances.
- **Clarifier des règles métier** : scénarios d'upgrade d'abonnement avec engagement en cours, validation des cartes (format, longueur).
- **Générer des exemples de tests manuels** : parcours d'onboarding, paniers multi-produits, création d'un programme puis duplication d'exercices dans une séance.
- **Montrer des patterns de code Python/Django** : snippets d'exemple pour structurer des vues, des formulaires et des validations, toujours relus avant intégration.

Les suggestions ont toujours été relues : aucune modification de schéma ou de logique critique n'a été appliquée sans validation humaine, pour éviter les hallucinations et conserver la compréhension du code.

4 Travail réalisé

4.1 Organisation

Notre méthode de travail : itérations courtes et binômes ciblés

Au démarrage, nous avons réparti les tâches par modules (authentification, programmes, boutique). Cette division claire a permis de livrer rapidement un squelette fonctionnel. Toutefois, dès que les règles métiers recoupaient plusieurs apps (ex. réutilisation du modèle utilisateur dans le panier ou le profil), nous avons travaillé en binôme sur les mêmes branches pour synchroniser les décisions de schéma et les migrations.

Nous avons privilégié des changements petits et fréquents, chaque fusion incluant une migration et un scénario manuel documenté. Les revues ont porté sur les validations de formulaires (en particulier l'upgrade d'abonnement et le calcul du total du panier) afin d'éviter les effets de bord.

Répartition fine des rôles :

- **Erick Jana Guana** : module suivi / journal d'entraînement, structuration des vues et des modèles de séances.
- **Sarah Metref** : conception des programmes et séances (formulaires, vues de création/édition/suppression, préremplissage des sets).
- **Yacine** : boutique, panier, historique de commandes et validation du checkout.
- **Salah-eddine Bekkari** : intégration transversale, uniformisation des templates, gestion Git, tests bout en bout et tests de non-régression.

Méthode initiale	Découpage par fonctionnalités isolées (auth, programmes, boutique) avec hypothèses locales sur les données utilisateur.
Limites constatées	Incohérences entre formulaires et modèles (ex. rôles non pris en compte dans le panier), migrations concurrentes difficiles à rebaser.
Méthode adaptée	Travail en binôme sur les flux transverses, relecture systématique des migrations, et usage de jeux de données minimaux pour rejouer les scénarios d'onboarding, d'upgrade et de paiement.

Résultat : cycles courts, moins de divergences de schéma, et une cohérence accrue entre le modèle utilisateur, les abonnements et la boutique.

4.2 Arborescence et principaux modèles

Structure du projet (ASCII)

```

FitTrackr/
|-- manage.py
|-- fixtures/
|-- media/
|-- static/
|   |-- css/
|   |   '-- base.css
|   |-- js/
|   |   |-- nav-dropdown.js
|   |   |-- onboarding-reminder.js
|   |   |-- progression.js
|   |   |-- chart.js
|   |   '-- flash-messages.js
|   '-- image/
|-- templates/
|   '-- base.html
'-- FitTrackr/
    |-- settings.py | urls.py | wsgi.py | asgi.py
    '-- app/
        |-- models.py | views.py | forms.py | urls.py | admin.py
        |-- management/commands/reset_demo_data.py
        '-- templates/
            |-- core/ (auth, abonnements, onboarding)
            |   |-- login.html | signup.html | subscriptions.html | onboarding.html
            |   |-- profile.html | admin_users.html | admin_user_edit.html | home.h
            |-- programs/ (programmes, séances, exercices)
            |   |-- program_list.html | program_detail.html | create_program.html
            |   |-- workout_list.html | workout_detail.html | create_workout.html
            |   |-- exercise_list.html | add_exercise_to_program.html | confirm_del
            |-- shop/ (produits, panier, commande, paiement)
            |   |-- product_list.html | product_detail.html | cart.html
            |   |-- checkout.html | checkout_success.html | order_history.html
            '-- suivi/ (badges, journal, progression)
                |-- dashboard.html | badges.html | journal.html | progression.html

```

Modèles clés

- **core.models.User** : custom AbstractUser avec rôles (member/coach/admin), email unique, métriques (âge, poids, taille), FK subscription. Proxy Coach et AdminUser pour filtrage naturel.
- **core.Subscription & SubscriptionEngagement** : plans tarifaires (prix, rang, engagement), historique des engagements avec dates calculées (compute_end_date) et propriété is_active.
- **core.Workout & WorkoutSet** : séances planifiées (date, durée, type) et séries associées (reps, poids, RPE, repos). Lien utilisateur obligatoire.
- **core.Exercise** : référentiel global d'exercices (muscle principal, équipement, difficulté).

- **core.Goal** : objectifs utilisateurs (poids cible, unités, statut) créés lors de l'onboarding.
- **programs.Program & ProgramExercise** : programmes créés par l'utilisateur (`created_by_user`) avec planification par jour/ordre et cibles (sets, reps, charge).
- **programs.Workout** (domaine programmes) : séances associées à un programme pour l'utilisateur, préremplies à partir de `ProgramExercise`.
- **shop.Product** : catalogue (nom, description, prix, stock, catégorie, image via `ImageField`).
- **shop.CartItem** : panier par utilisateur, quantité clampée ≥ 1 , total calculé côté serveur.
- **shop.Order, OrderItem, Payment** : commande (adresse, statut), lignes (quantité, prix unitaire) et paiement simulé (`PAY-{UUID}`, statut `paid/pending/failed`).
- **suivi.Badge & UserBadge** : récompenses associées aux utilisateurs, affichées dans `suivi/badges.html`.

4.3 Détail des pages applicatives (Vue par vue)

Accueil / Dashboard (`core/home.html`) : Point d'entrée après connexion, charge le contexte utilisateur (`request.user`) et expose des liens rapides vers `/programs`, `/shop`, `/subscriptions`. Le rendu est full server-side; aucune donnée sensible n'est injectée côté client. Les messages Django (`django.contrib.messages`) affichent l'état de l'onboarding et des changements d'abonnement. L'empreinte SQL est minimale (0 requête additionnelle, aucune préfetch nécessaire).

Inscription / Connexion (`core/signup.html`, `core/login.html`) : Formulaire basés sur `CustomerUserCreationForm` et `authenticate()`. La validation impose l'email unique et un mot de passe robuste (policy Django). En cas de succès, la session est régénérée et le flag d'onboarding est positionné (`request.session["onboarding_completed"] = False`). La protection CSRF est activée par défaut.

Onboarding (`core/onboarding.html`) : Formulaire `OnboardingForm` avec champs optionnels mais un contrôle applicatif `profile_done` qui déclenche une notification persistante tant que les données ne sont pas complètes. À la soumission, un objet `Goal` est créé (ou mis à jour) pour stocker l'objectif pondéral. Les conversions de date utilisent `timezone.now()` pour éviter les dérives de fuseau horaire.

Profil (`core/profile.html`) : Permet d'éditer email, username et métriques. Utilise `ProfileForm` et `update_fields` pour éviter les écritures inutiles. Les champs facultatifs sont normalisés (valeurs nulles admises) pour préserver la cohérence avec les calculs d'objectifs.

Abonnements (`core/subscriptions.html`) : Liste les offres `Subscription` ordonnées par niveau. La vue `subscriptions_view` calcule les dates de fin via `compute_end_date` (gestion des débordements de mois) et impose la règle métier : en présence d'un engagement actif, seules les upgrades sont autorisées. Les requêtes utilisent `select_related("sub` pour éviter le N+1 sur `SubscriptionEngagement`.

Programmes (`programs/program_list.html`, `program_detail.html`) : Restreints à l'auteur (`created_by_user=request.user`). `program_detail` précharge `program_exercises`

ordonnés (`day_index`, `order_index`). Les formulaires `ProgramForm` et `ProgramExerciseForm` valident les choix (niveau, objectif, charge cible) et créent les liens `ProgramExercise`.

Séances / Journal (`programs/workout_list.html`, `workout_detail.html`) : La vue `workout_list` sépare passé / futur via `workout_date <= today`. `workout_detail` charge soit les exercices du programme (via `program.program_exercises`) soit les sets saisis par l'utilisateur. La création (`create_workout`) clone les exercices du programme en `WorkoutSet` pour offrir un préremplissage des charges/répétitions.

Boutique (`shop/product_list.html`, `product_detail.html`) : Filtrage par catégorie (`cat`) en GET avec un index sur `category`. Les images produits sont servies via `ImageField` (stock local). `product_detail` vérifie l'existence du produit et expose le bouton d'ajout au panier.

Panier (`shop/cart.html`) : `CartItem` lié à l'utilisateur ; les quantités sont clampées à ≥ 1 . Chaque recalcul de total s'effectue côté serveur (pas de JS) pour éviter la fraude client. Les opérations utilisent `get_or_create` pour agréger les quantités et messages `.success` pour tracer l'état.

Checkout (`shop/checkout.html`) : Validation serveur des champs carte (longueur, digits, MM/AA, CVC). En absence de passerelle externe, un `Payment` est créé avec référence `PAY-{UUID}` et les `OrderItem` sont générés à partir des items du panier. Les vues sont protégées par login, et seules les requêtes internes peuvent déclencher la création de paiement.

Historique de commandes (`shop/order_history.html`) : Utilise `select_related("payment")` et `prefetch_related("items__product")` pour limiter les requêtes. Ordonné par `-created_at`. Permet de vérifier la cohérence paiement/commande sans exposer les données d'autres utilisateurs.

Administration interne (`core/admin_users.html`, `core/admin_user_edit.html`) : Restreint aux utilisateurs dont `role=admin`. Recherche multi-champs (`username`, `email`, `role`) et édition via `AdminUserForm`. Sécurisé par `PermissionDenied` en cas d'accès non autorisé.

5 Tests et non-régression

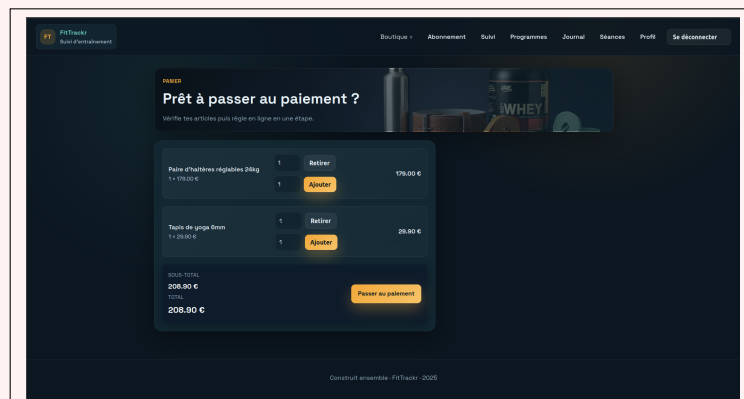
- **Tests unitaires ciblés** : vérification des helpers critiques (`compute_end_date`, validation carte, duplication `ProgramExercise` $\rightarrow$ `WorkoutSet`). Objectif : fiabiliser les règles métier sans dépendre du front.
- **Tests bout en bout manuels** : parcours d'onboarding complet, création programme + séance, ajout au panier + checkout. Chaque changement exige un scénario manuel documenté (étapes, données saisies, résultat attendu).
- **Non-régression (priorité)** : pour éviter de "massacrer" le travail des autres, chaque fonctionnalité modifiée doit rejouer les parcours critiques (auth, onboarding, abonnement, checkout). Les demandes de fusion ne sont validées que si les parcours existants restent verts ; en cas de changement de modèle, un replay systématique des migrations est imposé avant merge.

6 Difficultés rencontrées

Difficultés (Partie 1)

Principaux points de friction techniques :

1. **Custom User + migrations croisées** : l'extension d'AbstractUser (rôle, abonnements, métriques santé) a complexifié les migrations. Il a fallu verrouiller l'ordre des migrations (core avant shop) et régénérer celles qui dépendaient de la clé étrangère utilisateur. Alternative envisagée : un modèle de profil séparé, mais refusé pour éviter une jointure systématique.
2. **Onboarding partiellement rempli** : certains utilisateurs sautaient l'étape, provoquant des champs NULL dans les calculs d'objectifs. Correction : rendre les champs optionnels côté formulaire mais contrôler la complétude via `profile_done` et des messages contextuels; ajout d'un objectif par défaut lorsque le poids cible est absent. Une notification persistante reste affichée tant que les données santé ne sont pas fournies (rappel "intrusif" assumé car ces données conditionnent les programmes et le suivi).
3. **Cohérence panier / stock** : les quantités étaient décrémentées côté vue sans verrouillage. Solution minimale : recalcul systématique du total au chargement du panier et validation de la quantité avant création de `OrderItem`. Limite actuelle : pas de verrou pessimiste ni de gestion de concurrence forte; acceptable en mono-instance SQLite.



Difficultés (Partie 2)

4. **Validation du paiement** : le formulaire carte (checkout) devait rejeter les formats incorrects sans appel à une API externe. Implémentation : nettoyage des champs (longueur, chiffres, MM/AA) et production d'un identifiant pseudo-unique PAY-{UUID}. Limite : pas de 3-D Secure ni de webhooks ; une intégration Stripe est l'alternative naturelle.
5. **Fuseau horaire et dates d'entraînement** : l'utilisation de `timezone.now()` et des dates naïves entraînait des décalages sur les filtres "séances passées / à venir". Correction : normaliser en date locale (`timezone.now().date()`) et comparer via Django pour profiter des conversions de fuseau horaire.
6. **Performances des vues listes** : chargements multiples dans les vues `order_history` et `workout_list`. Optimisation : `select_related` sur paiement et abonnement, `prefetch_related` sur les items. Alternative : paginer les historiques ; jugé non prioritaire vu le volume actuel.
7. **Déploiement Render (<https://fittrackr-fgq9.onrender.com>)** : sur l'instance Render gratuite, le module de suivi (programs/workouts) ne fonctionne pas alors qu'il est opérationnel en local. Hypothèses : filesystem éphémère et endormissement du dyno entraînant des pertes de session ou de connexions SQLite. Pistes : passer à un plan avec disque persistant ou basculer sur PostgreSQL managé ; alternatives d'hébergement (Fly.io, Railway) envisagées.
8. **Gestion des images** : les uploads produits doivent conserver le type MIME et la taille raisonnable. Limite du stockage local : pas de redimensionnement automatique. Piste : utiliser Pillow pour générer des vignettes et préparer un stockage objet.
9. **Documentation des parcours** : plusieurs vues partagent des messages (upgrade, succès paiement). Nous avons factorisé les textes et ajouté des commentaires dans les vues pour éviter les divergences. Il reste à automatiser via un guide utilisateur centralisé.

7 Bilan

7.1 Conclusion

Enseignements clés

Le projet a consolidé notre pratique de Django : définition d'un modèle utilisateur personnalisé, mise en place d'un onboarding conditionnel, gestion d'abonnements avec logique métier, et intégration d'un mini tunnel d'achat. Nous avons mesuré l'intérêt du modèle de données Django pour maintenir la cohérence entre les apps (core, programs, shop) et de l'architecture "server-side rendering" pour sécuriser les flux sans JavaScript complexe. La discipline sur les migrations et la validation des formulaires a réduit les régressions et facilité les revues de code.

7.2 Perspectives

Améliorations envisagées

- **API REST + mobile** : exposer les programmes, séances et commandes via Django REST Framework pour une application mobile et une synchronisation multi-appareils.
- **Paieement réel et facturation** : intégrer Stripe (3-DS, webhooks) et générer des factures PDF, avec stockage des reçus.
- **Mesure et analytics** : stocker l'historique des charges et des temps de repos pour proposer des tableaux de bord (volumes, meilleurs records) et des alertes (surcharge).
- **Tests automatisés** : compléter la batterie de tests unitaires sur les vues sensibles (onboarding, checkout) et ajouter des tests d'intégration avec une base temporaire PostgreSQL.
- **Observabilité et performance** : cache (Redis) pour les listes de produits, compression des images, et instrumentation (logs structurés) pour les scénarios d'erreur.

FitTrackr nous a permis d'allier conception produit, règles métier et robustesse technique sur une stack Django moderne.

Salah-eddine Bekkari, Erick Jana Guana, Sarah Metref, Yacine

8 Webographie

Sources techniques mobilisées pour concevoir, sécuriser et étendre FitTrackr :

1. Documentation Django 5.0

Référence pour le modèle de données, les formulaires et la gestion des utilisateurs personnalisés.

2. Django Security

Checklist CSRF, XSS, sessions et bonnes pratiques pour les formulaires.

3. Stripe Payments

Référence pour une future intégration de paiements sécurisés et les webhooks.

4. Optimisation des requêtes Django

Bonnes pratiques pour `select_related/prefetch_related` et la pagination.

5. MDN Web Docs — Formulaires

Rappels sur l'ergonomie des formulaires et la validation côté client.

6. The Twelve-Factor App

Principes pour préparer le déploiement (config via variables d'environnement, logs standard).

7. Django Logging Best Practices

Préparation à l'observabilité et au suivi des erreurs en production.

9 Annexe

Cahier des charges

Attendus fonctionnels et techniques :

- **Gestion des comptes** : inscription, connexion, déconnexion et rôles multiples. Email unique et mot de passe sécurisé.
- **Onboarding santé** : collecte des mesures (âge, poids, taille) et objectif de poids ; création automatique d'un objectif associé.
- **Abonnements** : plans classés par niveau, engagements mensuels, upgrade contrôlé et historique d'engagements.
- **Programmes et séances** : création de programmes avec exercices, génération de séances pré-remplies, saisie des charges/répétitions, notes.
- **Boutique** : catalogue filtrable, panier par utilisateur, commande avec adresse, paiement simulé et historique des achats.
- **Contraintes techniques** : compatibilité Python 3.12, Django 5, stockage des images via Pillow, base SQLite en développement.

Exemple d'exécution

Exemple de déroulé complet :

1. **Lancement du serveur** : activation de l'environnement virtuel, exécution de `python manage.py migrate` puis `python manage.py runserver`. Les logs affichent le chargement des applications, la connexion SQLite et l'URL locale.
2. **Parcours utilisateur (exemple)** :
 - **Arrivée et inscription** : sur `/` ou `/signup/`, création du compte puis connexion via `/login/`. Les rôles sont gérés automatiquement (membre par défaut).
 - **Onboarding santé** : formulaire `/onboarding/` pour âge, poids, taille, objectif ; l'app calcule la cible et marque la session d'onboarding comme complétée.
 - **Choix d'abonnement** : page `/subscriptions/` listant les offres. En présence d'un engagement actif, seules les upgrades sont proposées et les dates de fin sont recalculées (`compute_end_date`).
 - **Création de programme** : sur `/programs/create/`, saisie du titre/objectif ; ajout d'exercices par jour sur `/programs/{id}/`. La génération de séance (`/workouts/create/`) clone les `ProgramExercise` en `WorkoutSet`.
 - **Saisie d'une séance** : page `/workouts/{id}/` pour renseigner charges, répétitions et RPE ; le journal (`/suivi/journal/`) liste les séances filtrables par dates.
 - **Suivi et badges** : tableau de bord `/suivi/dashboard/` (sessions, minutes, charge 4 semaines) et badges `/suivi/badges/` basés sur la régularité/volume.

- **Boutique et paiement** : catalogue /shop/, ajout au panier /shop/cart/, validation des quantités puis checkout /shop/checkout/. Un paiement simulé et une commande sont créés, consultables dans /shop/orders/.

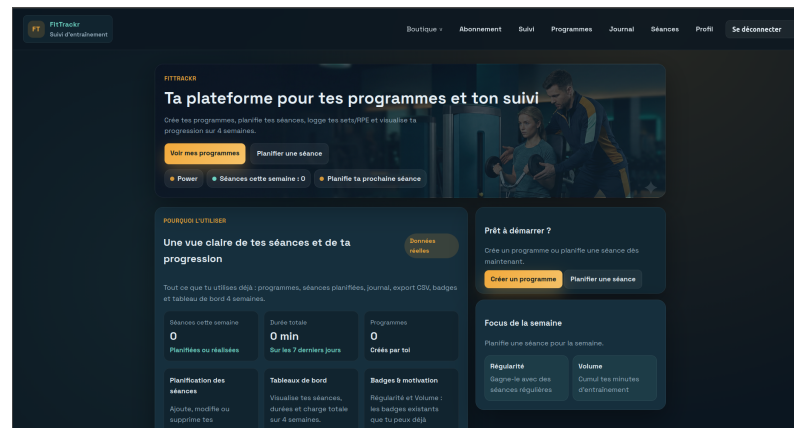


FIGURE 1 – Page d'accueil FitTrackr après connexion.

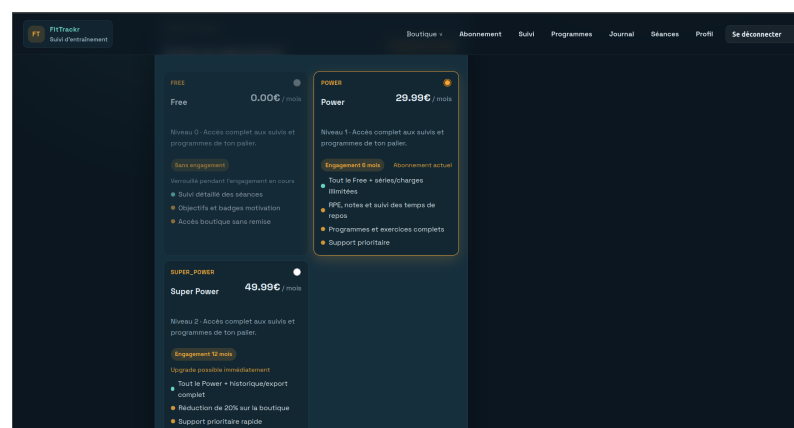


FIGURE 2 – Choix d'abonnement lors de l'onboarding.

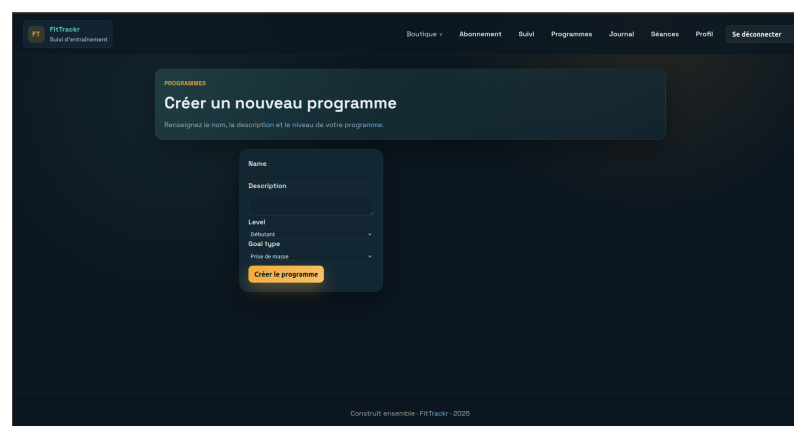


FIGURE 3 – Création d'un programme avec exercices.

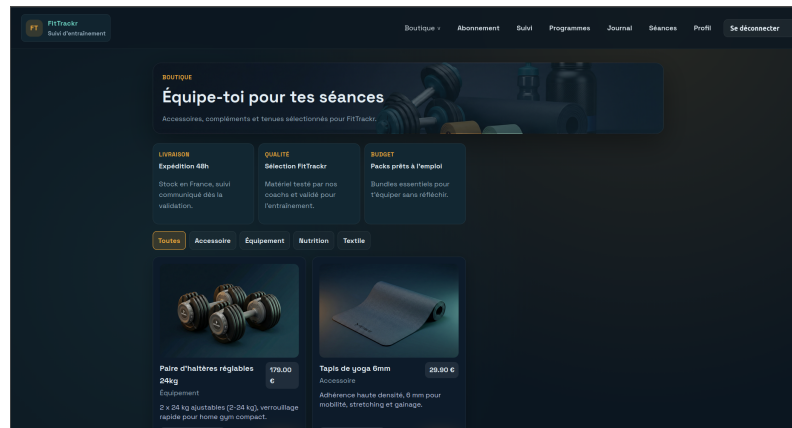


FIGURE 4 – Boutique et panier utilisateur.

Manuel utilisateur

Ce manuel décrit l'installation et les parcours principaux.

9.1 Installation

- Prérequis : Python 3.12, pip et virtualenv.
- Récupérer le projet :

```
git clone https://github.com/mbenzidane17/FitTrackr.git
cd FitTrackr
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser # facultatif pour l'admin
```

9.2 Lancement de l'application

- Démarrer le serveur de développement :

```
python manage.py runserver 0.0.0.0:8000
```
- Accéder à l'interface : `http://127.0.0.1:8000`. Le tableau de bord affiche la progression de l'onboarding et les liens vers programmes, séances et boutique.

9.3 Parcours utilisateur

- **Onboarding** : remplir âge, poids, taille et objectif. Le système crée un objectif `Goal` et marque la session comme complétée.
- **Programmes et séances** : créer un programme, ajouter des exercices par jour, puis générer une séance associée. Chaque `WorkoutSet` peut être ajusté (reps, charge, RPE).
- **Abonnements** : choisir une offre. En cas d'engagement actif, seuls les upgrades sont autorisés ; les dates de fin sont calculées par `compute_end_date`.
- **Boutique et paiement** : ajouter des produits, ajuster les quantités, valider l'adresse et les données carte (formats contrôlés). Le paiement génère un `Payment` et une commande archivée.

- **Administration** : via `/admin` ou la vue dédiée, un admin peut éditer les comptes, rôles et abonnements.